Product Requirements Document (PRD)

Project Title

Industrial Document Intelligence Platform

Overview

Industrial organizations store critical knowledge in technical manuals, maintenance guides, research reports, safety procedures, and engineering documentation.

Finding relevant information within hundreds or thousands of pages is time-consuming and inefficient.

This project aims to build a production-ready AI-powered document intelligence platform that enables semantic search, question answering, source-grounded responses, and multi-agent reasoning over technical documents.

The system will demonstrate modern AI Engineering skills including RAG, vector databases, agent systems, evaluation pipelines, API development, and containerized deployment.

Objective

Build a production-quality AI Engineering portfolio project that demonstrates:

- Retrieval-Augmented Generation (RAG)
- Vector Search
- Agent Workflows
- LLM Integration
- Evaluation Pipelines
- FastAPI Development
- Docker Deployment
- Production Software Architecture

The project should be suitable for showcasing AI Engineer skills to employers in Germany.

Problem Statement

Engineers and researchers spend significant time searching through:

- Technical manuals
- Research papers
- Safety documentation
- Product specifications
- Maintenance guides

Traditional keyword search:

- Does not understand meaning
- Misses relevant information
- Produces poor search experiences

A semantic retrieval and reasoning system is needed to improve knowledge access.

Target Users

Primary Users:

- AI Engineers
- Software Engineers
- Researchers
- Technical Teams

Secondary Users:

- Manufacturing Organizations
- Industrial Automation Teams
- Robotics Companies
- Engineering Departments

Core Features

1. Document Upload

Supported Formats:

- PDF
- DOCX

- TXT

Requirements:

- Single document upload
- Multiple document upload
- Metadata extraction
- Duplicate detection
- File validation

---

2. Document Processing Pipeline

The system should:

- Extract text
- Clean text
- Chunk documents
- Generate embeddings
- Store metadata
- Store vectors

Metadata:

- Document name
- Upload timestamp
- File type
- Page number
- Chunk ID

---

3. Vector Search

Capabilities:

- Semantic similarity search
- Top-k retrieval
- Metadata filtering
- Relevance scoring

Vector Database:

- Qdrant

---

4. Retrieval-Augmented Generation

Workflow:

User Question

↓

Query Embedding

↓

Vector Retrieval

↓

Context Assembly

↓

LLM Response

↓

Citation Generation

Requirements:

- Grounded responses only
- Context-aware answers
- Citation support
- Hallucination reduction

---

5. Multi-Agent Architecture

Agent Framework:

- LangGraph

Agents:

Retrieval Agent

Responsibilities:

- Retrieve relevant chunks
- Rank search results

Reasoning Agent

Responsibilities:

- Analyze retrieved context
- Generate answer

Verification Agent

Responsibilities:

- Check answer consistency
- Detect unsupported claims

Citation Agent

Responsibilities:

- Attach source references
- Generate citation metadata

---

6. Source Citations

Every response should include:

- Document name
- Page number
- Chunk reference

Example:

Source: Maintenance_Manual.pdf Page 42

---

7. Evaluation Pipeline

Evaluation Framework:

- RAGAS

Metrics:

Faithfulness

Measures whether answers are supported by retrieved context.

Context Recall

Measures retrieval quality.

Answer Relevancy

Measures usefulness of responses.

Requirements:

- Automated evaluation scripts
- Benchmark dataset
- Reproducible evaluation workflow

---

8. REST API

FastAPI Endpoints:

POST /documents/upload

GET /documents

DELETE /documents/{id}

POST /chat/query

GET /health

GET /metrics

Non-Functional Requirements

Performance

Document Upload:

- Less than 10 seconds for typical files

Question Answering:

- Less than 5 seconds response time

Reliability

- Error handling
- Input validation
- Logging
- Graceful failure recovery

Scalability

Architecture should support:

- Thousands of documents
- Multiple collections
- Future model upgrades

Maintainability

Requirements:

- Modular codebase
- Type hints
- Unit tests
- Clear folder structure

Technology Stack

Backend:

- FastAPI

Agent Framework:

- LangGraph

Vector Database:

- Qdrant

LLM:

- Llama 3 via Ollama

Embeddings:

- BGE-Small or
- Nomic Embed

Evaluation:

- RAGAS

Containerization:

- Docker
- Docker Compose

Testing:

- Pytest

Code Quality:

- Ruff
- Black

Python:

- Python 3.12

Success Metrics

Technical Metrics:

- Citation coverage > 95%
- Retrieval accuracy > 80%
- Average response latency < 5 seconds
- Successful document ingestion > 95%

Evaluation Metrics:

- Faithfulness > 0.80
- Context Recall > 0.75
- Answer Relevancy > 0.80

Portfolio Value

This project should demonstrate:

- FastAPI
- LangGraph
- RAG
- Vector Databases
- Qdrant
- Ollama
- Open-source LLMs
- Agent Systems
- Evaluation Pipelines
- Docker
- AI System Design
- Production Architecture

The final project should be suitable for inclusion on a resume targeting AI Engineer, Applied AI Engineer, GenAI Engineer, and Machine Learning Engineer positions.